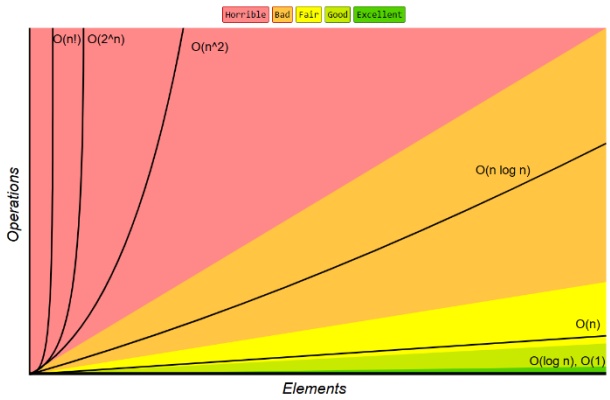
OPERATING SYSTEMS (UE24CS242B)

Unit 1 – Kernel Data Structures & Computing Environments

Kernel Data Structures

Kernel data structures are fundamental building blocks used internally by the operating system to manage processes, memory, devices, and resources efficiently. Since the kernel operates under strict performance and reliability constraints, the choice of data structures directly impacts OS efficiency.



Common Data Structure Operations

Data Structure	Time Complexity							
	Average				Worst			
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion
Array	Θ(1)	Θ(n)	Θ(n)	Θ(n)	O(1)	O(n)	O(n)	O(n)
Stack	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)
Queue	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)
Singly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)
Doubly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)
Skip List	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)
Hash Table	N/A	Θ(1)	Θ(1)	Θ(1)	N/A	O(n)	O(n)	O(n)
Binary Search Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(n)	O(n)	O(n)
B-Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))
Red-Black Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))
Splay Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))
AVL Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))
KD Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)

Arrays

An array is a simple and fundamental data structure where:

- Elements are stored in contiguous memory locations
- Each element can be accessed directly using an index

Access Mechanism

For items occupying multiple bytes:

- Address = *base address + (item number × item size)*

Limitations of Arrays

- Cannot efficiently store items of **variable size**
- Insertion and deletion are costly because:
    - Relative order must be preserved
    - Shifting of elements is required

Due to these limitations, arrays are not always suitable for dynamic kernel data management.

Lists

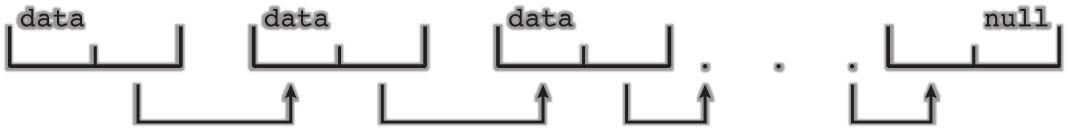
Operating systems extensively use **standard programming data structures**, especially **lists**, to handle dynamic data.

Singly Linked List

- Each element (node) contains:
    - Data

- Pointer to the **next** element

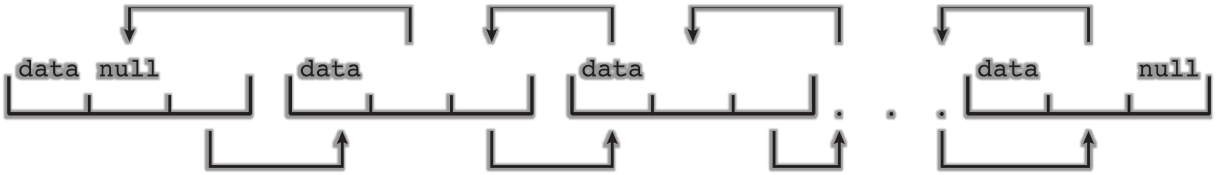
- Items must be accessed in a particular order



Types of Linked Lists

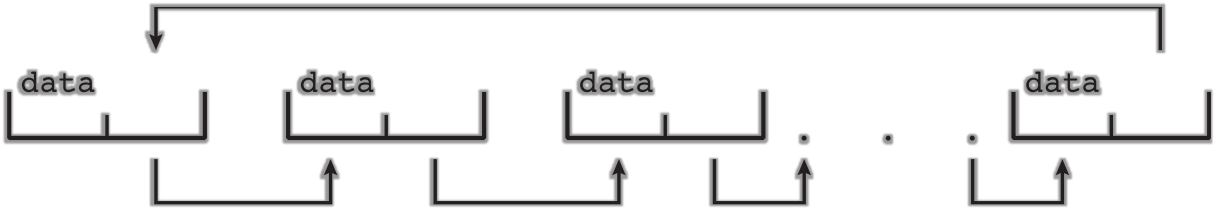
Doubly Linked List

- Each node contains:
  - Pointer to the previous node
  - Pointer to the next node
- Allows traversal in both directions



Circular Linked List

- The last node points back to the first node
- No NULL pointer at the end



Linked Lists – Advantages and Disadvantages

Advantages

- Can store items of **varying sizes**
- **Easy insertion and deletion**
- No need for contiguous memory allocation

Disadvantages

- Retrieving a specific element takes **O(n)** time
- Sequential traversal required

Usage in OS

- Used directly by kernel algorithms
- Used to build other structures such as:
    - Stacks
    - Queues

Stacks and Queues

Stack

A **stack** is a sequential data structure that follows the **LIFO (Last-In, First-Out)** principle.

Usage in OS

- Used during **function calls**
- Stores:
    - Function parameters
    - Local variables
    - Return address
- On function call → data is **pushed**
- On function return → data is **popped**

Queue

A **queue** is a sequential data structure that follows the **FIFO (First-In, First-Out)** principle.

Usage in OS

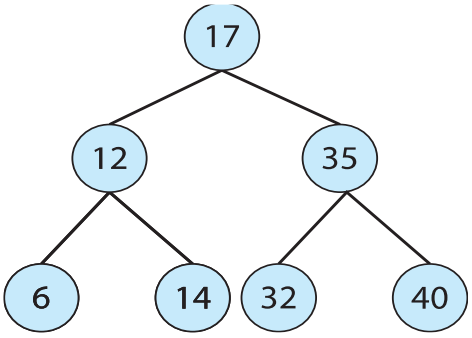
- Processes waiting for CPU execution
- Print jobs sent to printers
- Disk I/O requests

Trees

Trees are data structures used to represent **hierarchical relationships**.

Key Characteristics

- Parent–child relationship between nodes
- Commonly used form: **Binary Search Tree (BST)**



Binary Search Tree Property

- Left child ≤ parent
- Right child > parent

Performance

- Unbalanced BST: **O(n)**
- Balanced BST:

- Height ≤ log n
- Search time **O(log n)**

Usage in OS

- Linux uses balanced trees for **CPU scheduling**
- Helps efficiently select the next process to run

Hash Functions and Maps

A **hash function** maps input keys to output values.

Key Points

- Two different inputs may produce the same output (collision)
- Used to implement **hash maps**

Hash Map

- Stores **key–value pairs**
- Provides very fast lookup
- Average search time: **O(1)**

Usage in OS

- Process lookup
- Resource mapping
- Fast access tables

Bitmaps

A **bitmap** is a string of binary digits used to represent the status of resources.

Representation

- 0 → Resource available
- 1 → Resource unavailable

The **i^{th} bit** corresponds to the **i^{th} resource**.

Example

Bitmap: 001011101

- Unavailable resources: 2, 4, 5, 6, 8
- Available resources: 0, 1, 3, 7

Usage in OS

- Disk block allocation
- Memory frame allocation

- Efficient for managing a large number of identical resources

Computing Environments – Traditional

Traditional computing environments include:

- Stand-alone, general-purpose machines

However, boundaries are blurred due to:

- Internet connectivity
- Networked systems

Characteristics

- Web portals for internal access
- Thin clients (network computers)
- Home systems using **firewalls** for security

Computing Environments – Mobile

Mobile computing includes:

- Smartphones
- Tablets
- Handheld devices

Differences from Traditional Laptops

- Additional OS-level features:
    - GPS
    - Gyroscope
- Enables new applications such as:
    - Augmented Reality

Connectivity

- IEEE 802.11 (Wi-Fi)
- Cellular networks

Leading Platforms

- Apple iOS
- Google Android

Computing Environments – Distributed

Distributed computing consists of:

- Multiple independent systems
- Connected via a network

Network Types

- LAN (Local Area Network)
- WAN (Wide Area Network)
- MAN (Metropolitan Area Network)
- PAN (Personal Area Network)

Characteristics

- Uses TCP/IP
- Network Operating System coordinates communication
- Message passing between systems
- Provides illusion of a single unified system

Computing Environments – Client–Server

In client–server computing:

- Clients send requests
- Servers respond with services

Types

1. **Compute-Server System**
    - Provides computational services
    - Example: Database server
2. **File-Server System**
    - Manages file storage and retrieval

This model replaced earlier dumb terminals with intelligent clients.

Computing Environments – Peer-to-Peer (P2P)

In P2P systems:

- No distinction between client and server
- All nodes are peers
- Each node can act as:
    - Client
    - Server
    - Both

Operation

- Node joins P2P network
- Registers services or uses discovery protocols

Examples

- Napster
- Gnutella
- VoIP systems like Skype

Computing Environments – Virtualization

Virtualization allows:

- Multiple operating systems to run on the same hardware

Techniques

1. **Emulation**
    - Different CPU architectures
    - Slowest method
2. **Interpretation**
    - Code not compiled to native instructions
3. **Virtualization**
    - Guest OS compiled for same CPU
    - Efficient and widely used

Virtual Machine Manager (VMM)

- Provides virtualization services
- Manages guest operating systems

Uses of Virtualization

- Running multiple OSes on one system
- Application compatibility testing
- Software development and QA testing
- Data center resource management

VMM can also act as the host OS.

Computing Environments – Cloud Computing

Cloud computing combines:

- Traditional operating systems
- Virtual Machine Managers
- Cloud management tools

Characteristics

- Internet-based access

- Load balancers distribute traffic
- Firewalls ensure security

Cloud Computing Models

Cloud delivers computing resources as services:

1. **Software as a Service (SaaS)**
    - Applications delivered via Internet
    - Example: Online word processors
2. **Platform as a Service (PaaS)**
    - Complete development platform
    - Example: Database platforms
3. **Infrastructure as a Service (IaaS)**
    - Virtual servers and storage
    - Example: Backup storage services

Cloud Types

- Public cloud
- Private cloud
- Hybrid cloud

Computing Environments – Real-Time Embedded Systems

Real-time embedded systems are:

- The most common form of computers
- Special-purpose
- Limited functionality

Characteristics

- Often use **Real-Time Operating Systems (RTOS)**
- Must meet **strict time constraints**
- Correct operation depends on meeting deadlines

Some embedded systems:

- Use lightweight OS
- Some operate without an OS at all